Database modelling

MBUA 512

Class 2 – where we're going

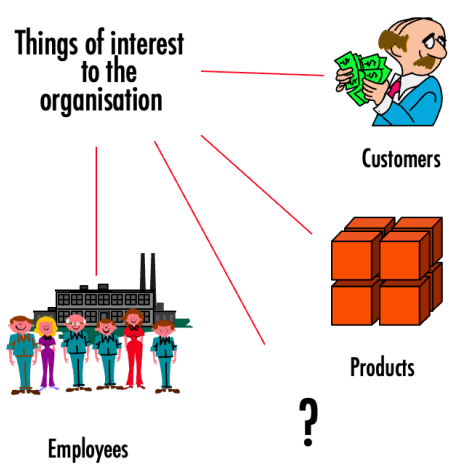
Database modelling, with Michael Winikoff

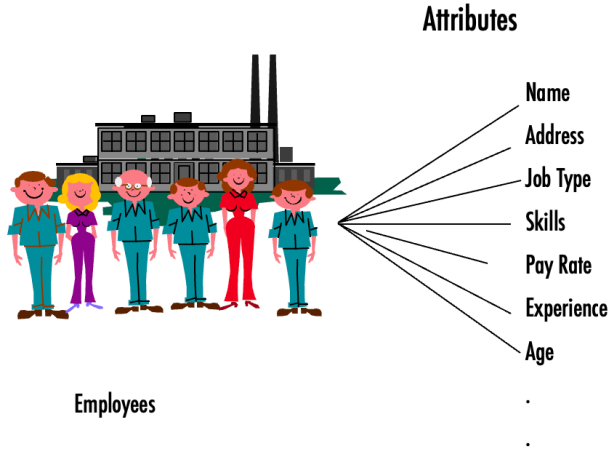
1. Data modelling using diagrams
2. Translating diagrams to relational databases, and the role of *keys*
3. Anomalies, and avoiding them by *normalisation*

Based on slides by Professor Markus Luczak-Roesch.

Recap from last week

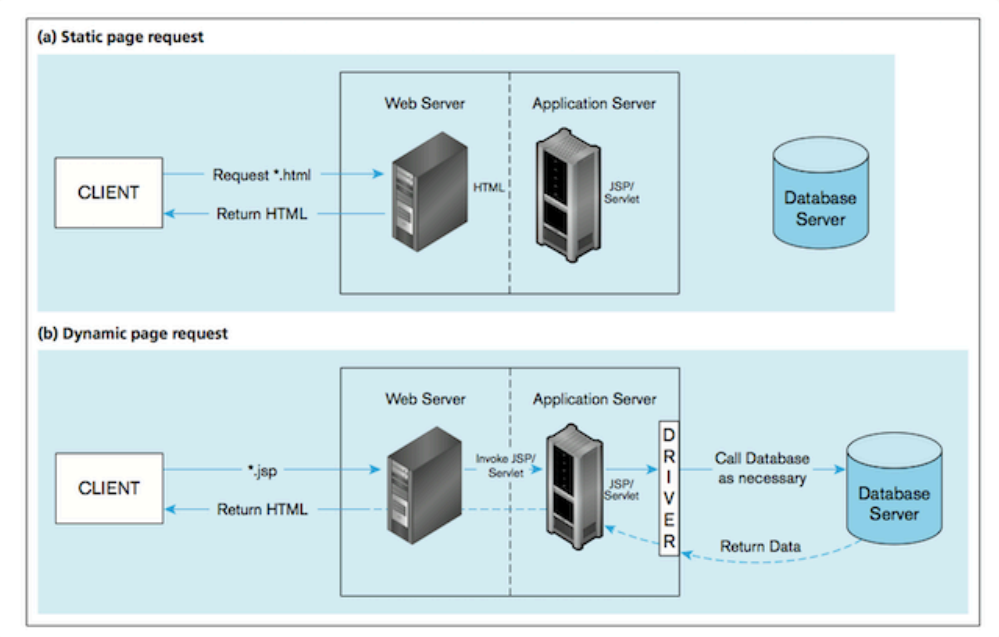
- What is data?
  - Types of data (qualitative/quantitative, structured/unstructured)
  - Why is data important … to organisations? … to business analysts?
- What is a DB management system and what is a *relational* database? (technology)
- Concepts: **entity**, **attributes**, **relationships** (modelling data)





Big picture

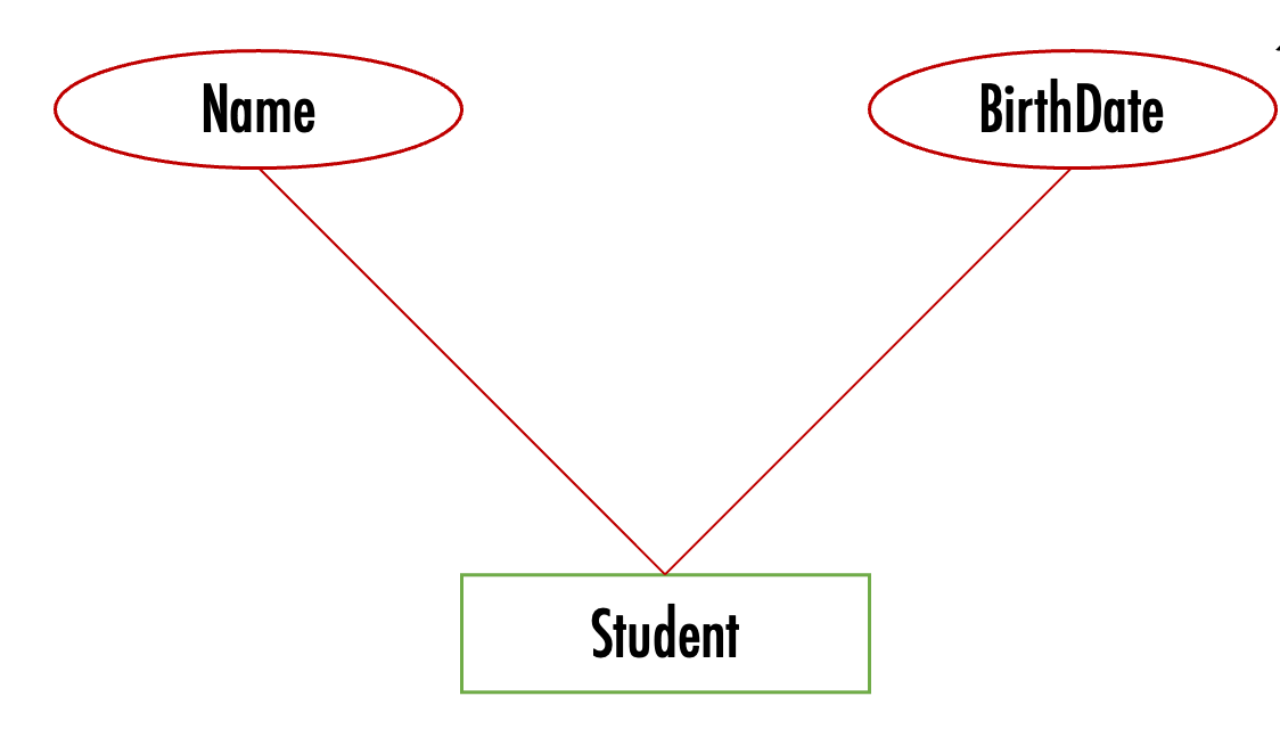
Roles: DB analysis, designers, developers, front-end app analysts and developers, DB admin



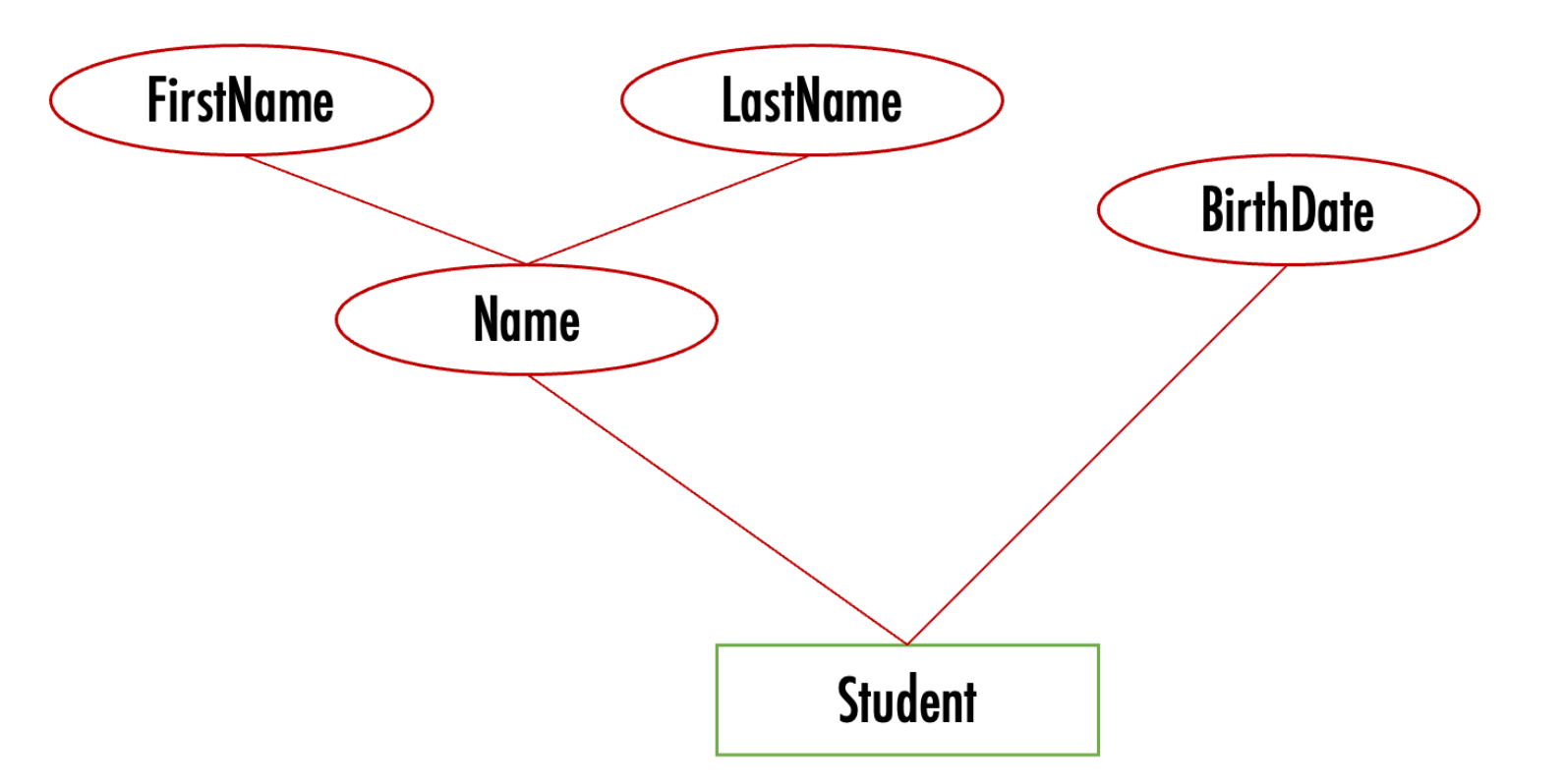
1. Data modelling using diagrams

Model *conceptual* model in terms of entities, attributes, relationships using an **ER (Entity-Relationship) diagram**:

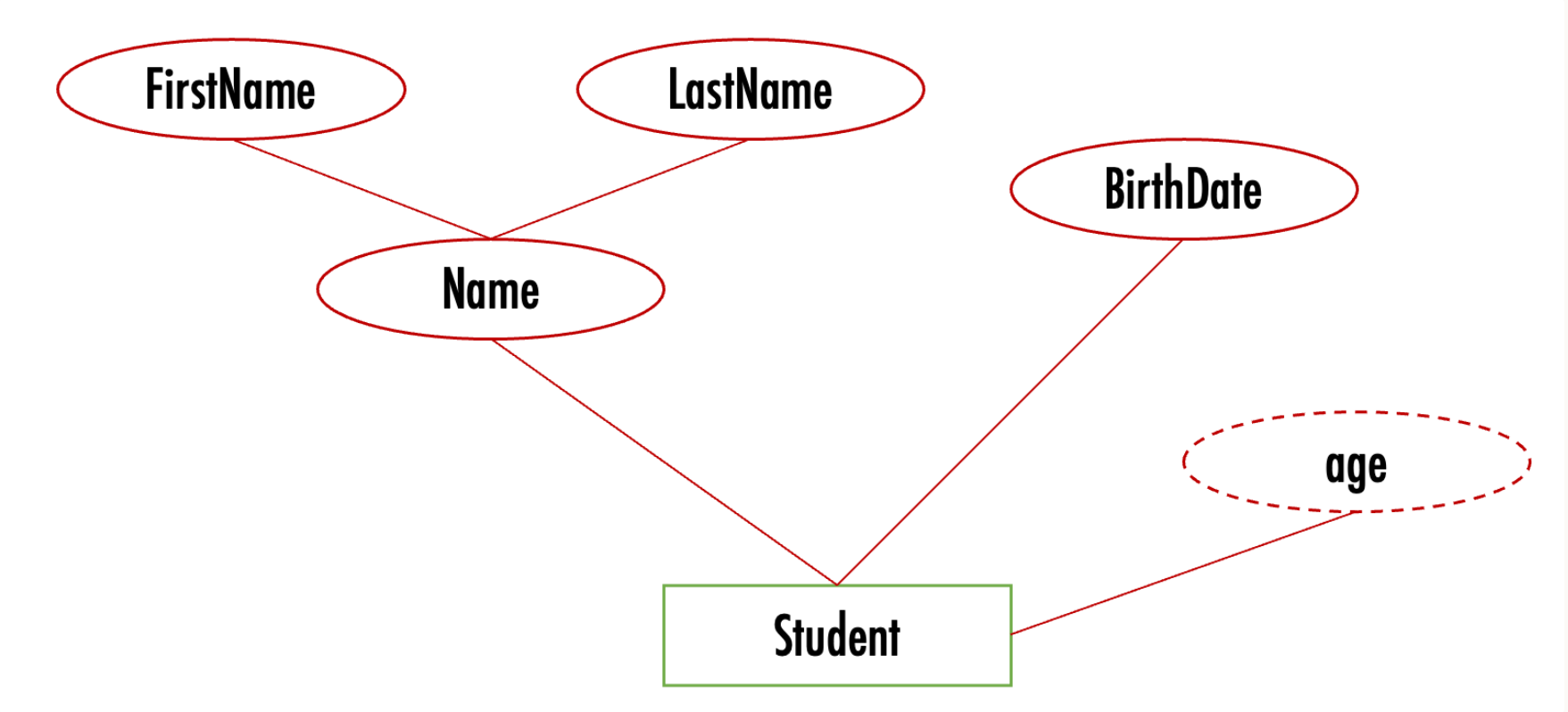
- Entity = rectangle
- Attribute = oval
- Relationship = Diamond



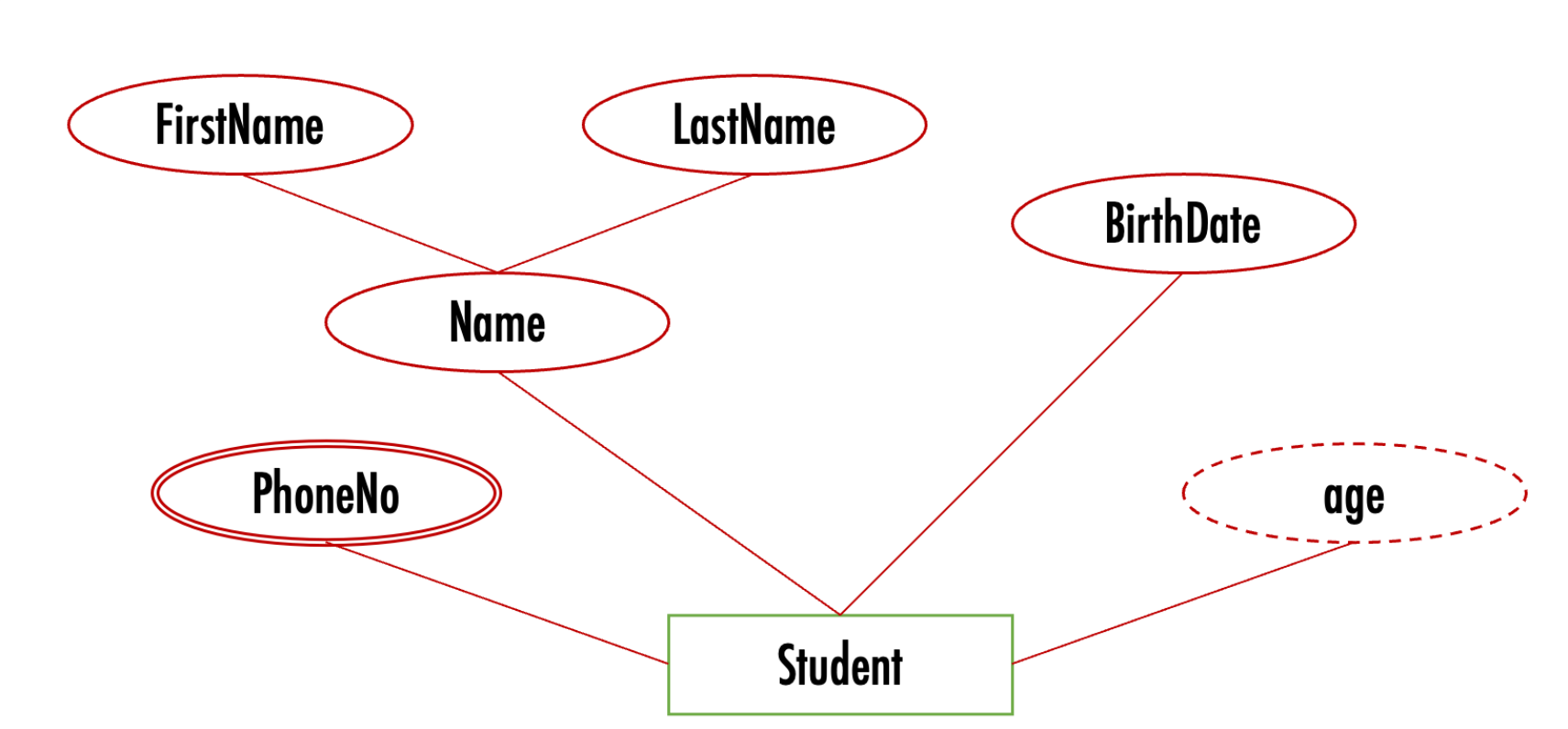
Attributes can be *composite*



Attributes can be *derived* (dashed oval)



Attributes can have *multiple values* (double edged oval)

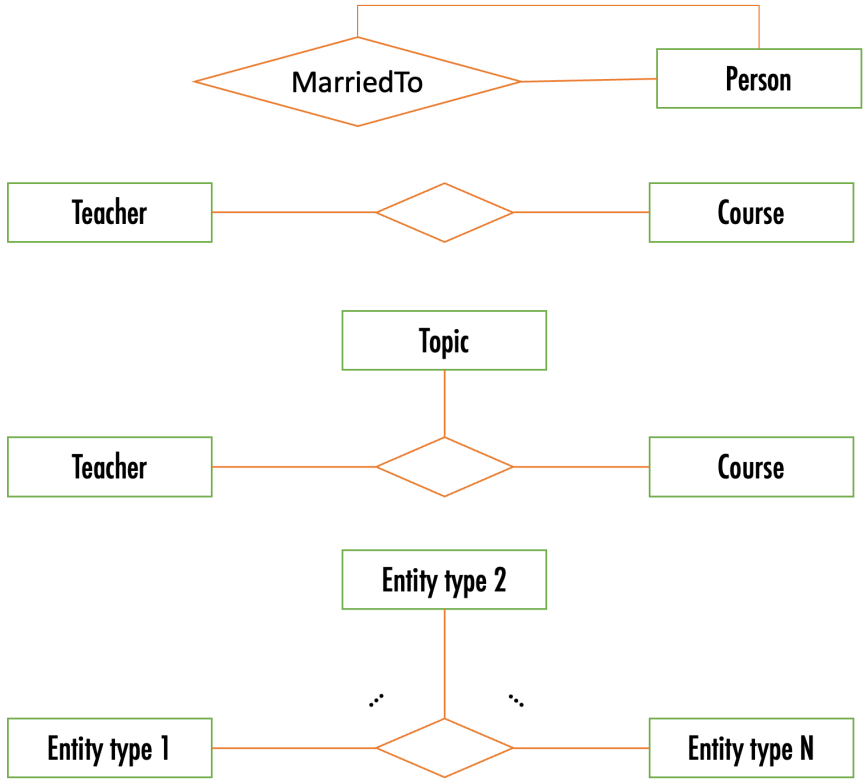


Relationships ...

- A relationship links one or more entities
- Relationships have **degree** and **cardinality**:
  - Degree: number of participating entity **types** in a relationship
  - Cardinality: number of entity **instances**
- Example:
  - MarriedTo is Unary degree: only one type of entity is involved: a Person
  - MarriedTo is 1-1 cardinality: each individual person is married to one other person
- Example:
  - Teaches is Binary degree: two entity types, teacher and course
  - Teaches is N-N cardinality: a teacher can teach many courses, and a course can have many teachers

Degree – number of types of entities

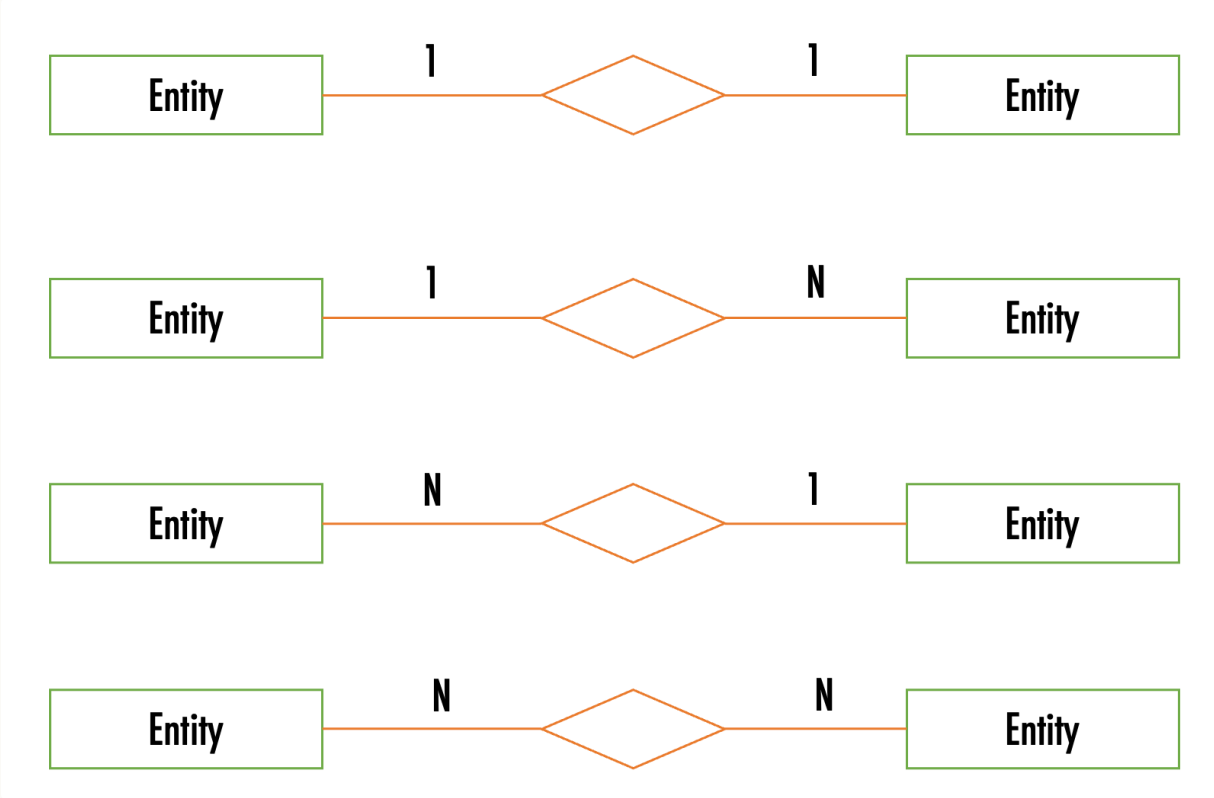
- Unary
- Binary
- Ternary
- n-ary



Cardinality – number of *instances* of entities

Left: one-to-one (e.g. bus and bus driver); Right: one-to-many (e.g. bus and passenger)

Notation for cardinality

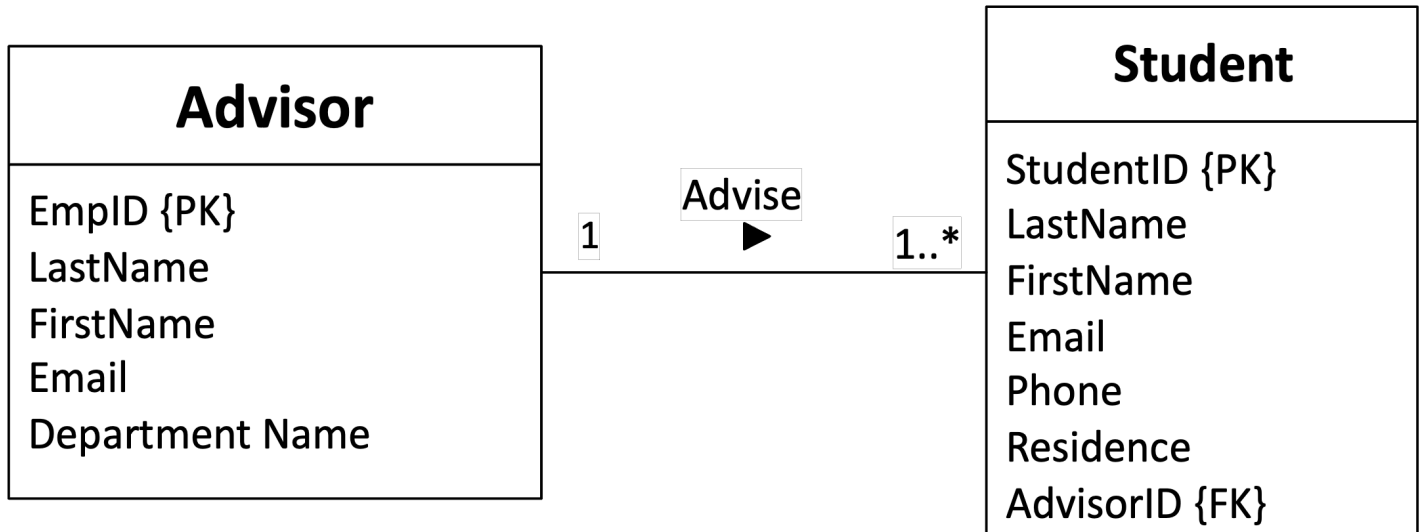


Keys

- A **key** is one or more attributes that must be unique to each entity instance, i.e. no two instances can have the same value for their key attribute(s)
  - Example: name is a bad choice for key - two people can have the same name
- A *super key* is a set of attributes that can be a key
- A *candidate key* is a *minimal* super key (no subset of it is a super key)
- A *primary key* is a chosen candidate key
- Notation: key attribute is <u>underlined</u>

Alternative notation: UML (Unified Modeling Language)

- Rectangle = Entity, with attributes listed in the rectanagle
- Relationship = line, with cardinalities indicated using annotation at each end (e.g. "1", "0..", "1.." - "*" means "many")
- Keys indicated with `{PK}`

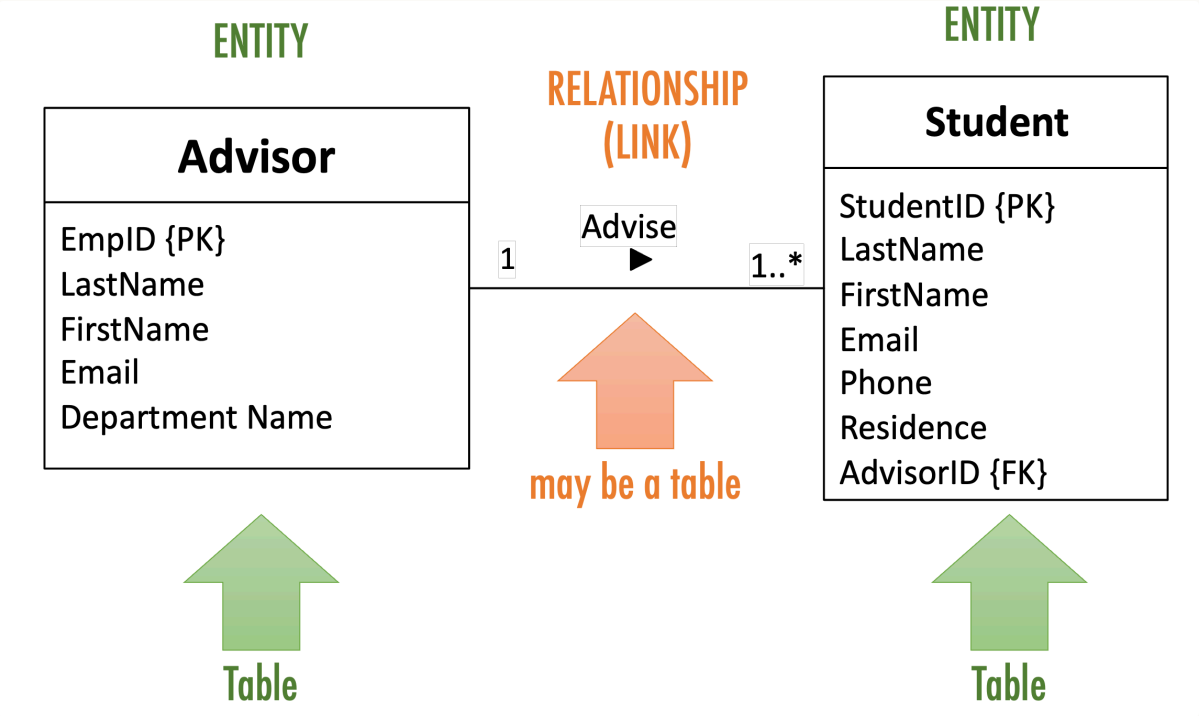


Tips for modelling

- Iterative!
- Need to focus on what is important, rather than capture everything
- Naming:
  - Brief, but clear
  - Unique - e.g. don't use "name" for both "student name" and "employee name"
  - Entities and attributes: singular ("student")
  - Relationships: verbs or verb phrases ("inspects", "manages", "belongsTo")

2. Translating diagrams to relational databases

- Each entity becomes a table (relation) containing the entity's attributes
- Each relationship is mapped to either a table or additional attributes, depending in the cardinalities



Mapping a 1-N relationship as an additional attribute

Mapping an N-N relationship as a table

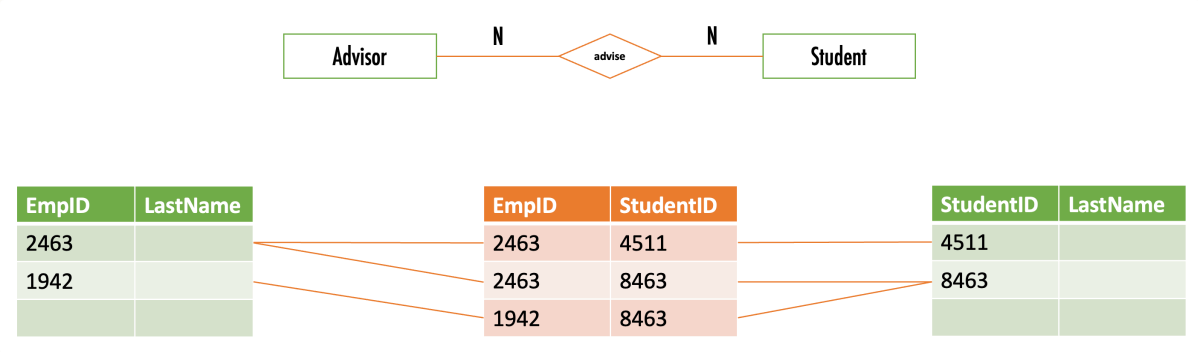
EmpID	LastName
2463	
1942	

EmpID	StudentID
2463	4511
2463	8463
1942	8463

StudentID	LastName
4511	
8463	

Foreign keys

- A *key* is the attribute(s) that can be used to identify an entity instance (e.g. StudentID)
- Each table (relation) will have column(s) for its keys (e.g. the Student table has a column StudentID)
- A *foreign key* is when a table has a column that contains keys for *another table* (e.g. Student table contains the ID of the student's advisor)
  - in UML notation indicated with `{FK}`



3. Anomalies and Normalisation

Errors or inconsistencies occurring when updating tables with redundant data:

- insertion anomaly example: require data about different entities to be entered at the same time in a single relation – cannot track customer address separately from order
- deletion anomaly example: deletion of a row results in loss of multiple facts – deleting the last order loses the customer's contact details!
- modification anomaly example: need to modify multiple rows to update a fact consistently – update address requires updating every order

Example on the next slides.

Solution: *normalise* by splitting up the table

How to do this? By analysing *functional dependencies*.

Functional dependency - occurs when the value of one (or more) column(s) in each record of a relation uniquely determines the value of another column in that same record of the relation

Example: ClientID → ClientName

Dependencies for the ad campaign DB

Types of functional dependencies

- ❌ **Partial**: a column depends on a *component* of a composite primary key; e.g. ModusID → Media, Range
- ✅ **Full**: the primary key functionally determines the column, and no separate component of it partially determines the same column
- ❌ **Transitive**: nonkey columns functionally determine other nonkey columns; e.g. CampaignMgrID → CampaignMgrName

Dependencies for the ad campaign DB

Normalisation - 1NF

- First Normal Form (1NF): no multi-value entries
- Normalise by splitting records

Normalisation - 2NF

- Second Normal Form (2NF): a table is in 2NF if it is in 1NF and contains no **partial** functional dependencies — one way to ensure this is a single-column primary key
- Normalise by splitting out things that depend on part of the key into a separate table

Normalisation - 3NF

- Third Normal Form (3NF): A table is in 3NF if it is in 2NF and if it does not contain transitive functional dependencies
- Normalise by splitting out things that depend on nonkey columns into a separate table

Result on the next slide.

Normalisation Summary

- Having too many things (columns) in a table can cause anomalies when adding/deleting/changing records
- This occurs when there are partial or transitive functional dependencies
- Solution is to normalise to 3NF:
  1. Identify functional dependencies
  2. Split out columns to remove partial and transitive functional dependencies

This is probably the trickiest database concept we cover!

Good news: typically a database that results from a design process (identifying entities, attributes, relationships) is in 3NF

Today

1. Data modelling using ER diagrams
2. Translating diagrams to relational databases, and the role of *keys*
3. Anomalies, and avoiding them by *normalisation*

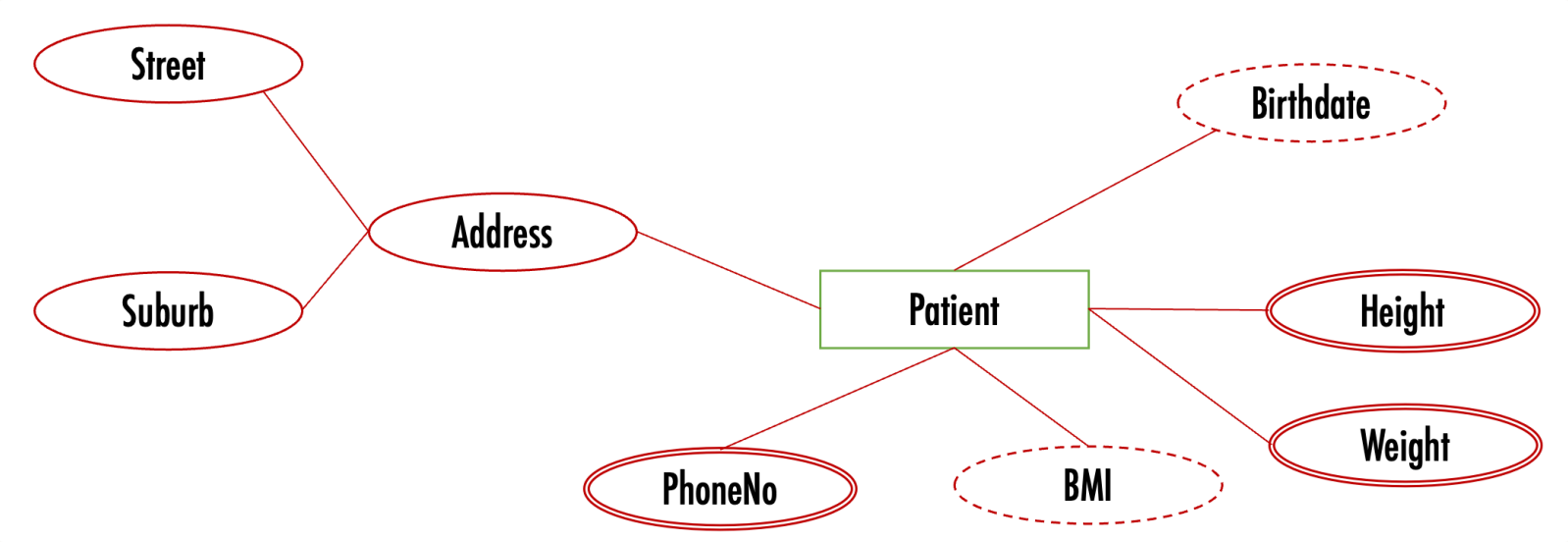
Next: activities in part 2 today – learn by doing. Next week, building, modifying and querying relational databases with SQL ("es-queue-el", or "sequel").

Activities

Activity – interpret an ER diagram

~ 10 minutes, in groups

- What does the following ER diagram say?
- How might you make it more accurate?



Activity — sketch an ER diagram

~ 15 minutes, in groups

- **Entities:** Student, Degree, Course, University
- **Relationships:** EnrolledIn, OffersDegree, GraduatedWith

Sketch the ER diagram. What are the **degree** and the **cardinality** of each relationship?

Activity – sketch it in UML notation

~5 minutes, in groups

Sketch a UML diagram showing the same information as the ER diagram you prepared
earlier.

- **Entities:** Student, Degree, Course, University
- **Relationships:** EnrolledIn, OffersDegree, GraduatedWith

Activity – normalisation

~15 minutes, in groups

- Identify the dependencies in the relation on the next slide – assume the primary key is Flight Number and Day.
- How would you transform it into 3NF? Identify functional dependencies, classify them, and split to remove partial and transitive dependencies.

Assumptions: a flight number always flies at the same time; a flight number does not always use the same plane model. What else do you need to assume?

Flight Number	Airline	Day	Time	Pilot ID	Pilot Name	Plane model	Plane manufacturer
NZ123	Air NZ	10 Aug 2018	10am	PL8715	Pat Smith	Airbus310	AirBus
NZ456	Air NZ	11 Aug 2018	11am	PL8716	Yi Zhang	Airbus320	AirBus
YA789	Yugoslav Air	17 Aug 2018	10:30am	PL9781	Jim Belich	Boeing737	Boeing
NZ123	Air NZ	12 Aug 2018	10am	PL9781	Jim Belich	Boeing737	Boeing
NZ124	Air NZ	13 Aug 2018	11pm	PL9781	Jim Belich	Airbus310	Airbus
YA890	Yugoslav Air	17 Aug 2018	4pm	PL9781	Jim Belich	Airbus310	Airbus
…	…	…	…	…	…	…	…